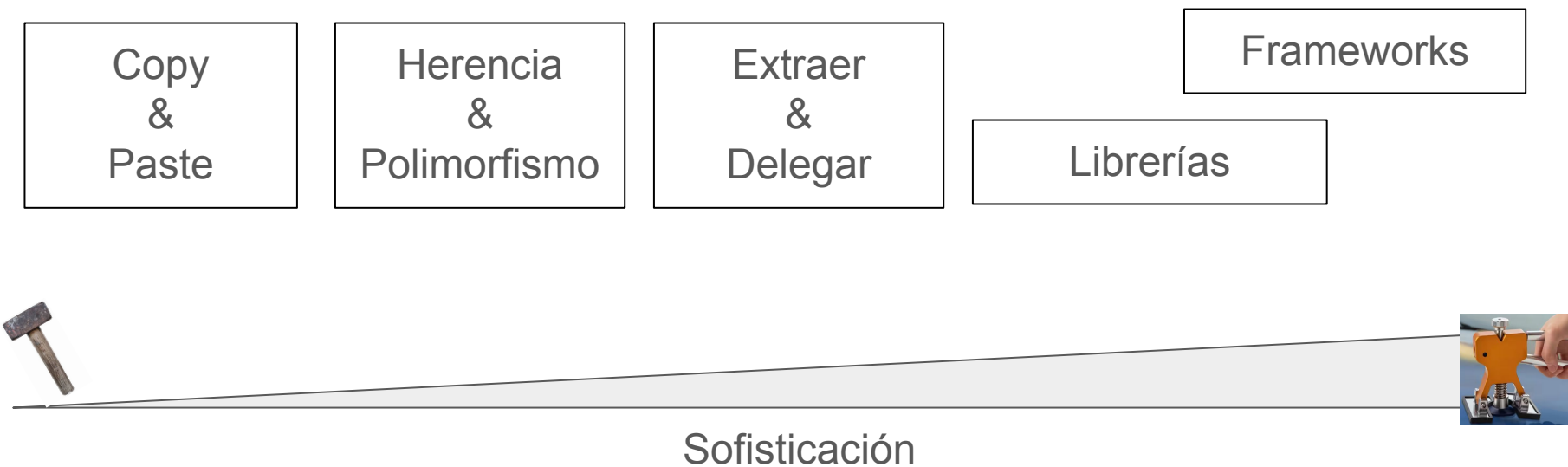


El reuso es inesperado



Modos de reuso



Frameworks I

Federico Balaguer

Definiciones

Un framework es una aplicación “semi-completa”, “reusable”, que puede ser especializada para producir aplicaciones a medida...

...un conjunto de clases concretas y abstractas, relacionadas para proveer una arquitectura reusable que implementa una familia de aplicaciones (relacionadas)...

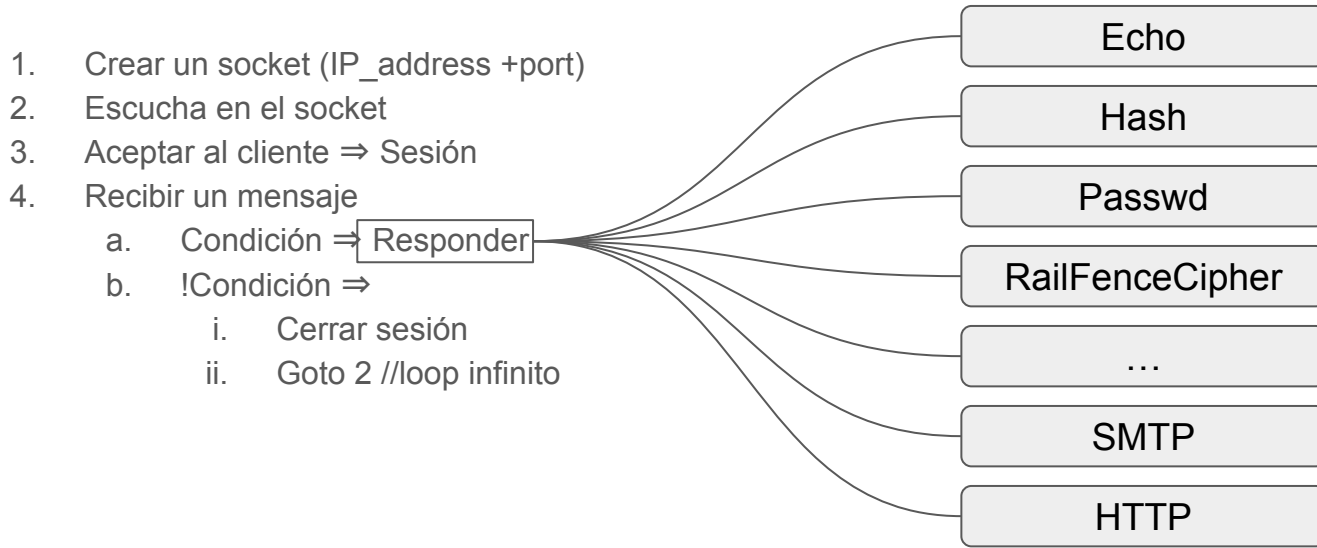
Foco del diseño

- Una aplicación está diseñada para responder a requerimientos de usuarios
- Una librería está diseñada para proveer reuso de cierto tipo de funcionalidad (código fuente). Nuestro código “llama” o “invoca” funcionalidad de la librería
 - Autenticación
 - Cliente LLM
 - Conectividad a Base de Datos
- Un framework está diseñado para proveer reuso de una manera de ejecutar (arquitectura). Se provee código o se configura para que ejecute
 - Application Frameworks (tienen **loop** de control)
 - Integration / Infrastructure Frameworks
 - En todos los casos tiene un método (hilo) que implementa “inversion de control” (Hollywood Principle)

Aplicación: Servidores TCP

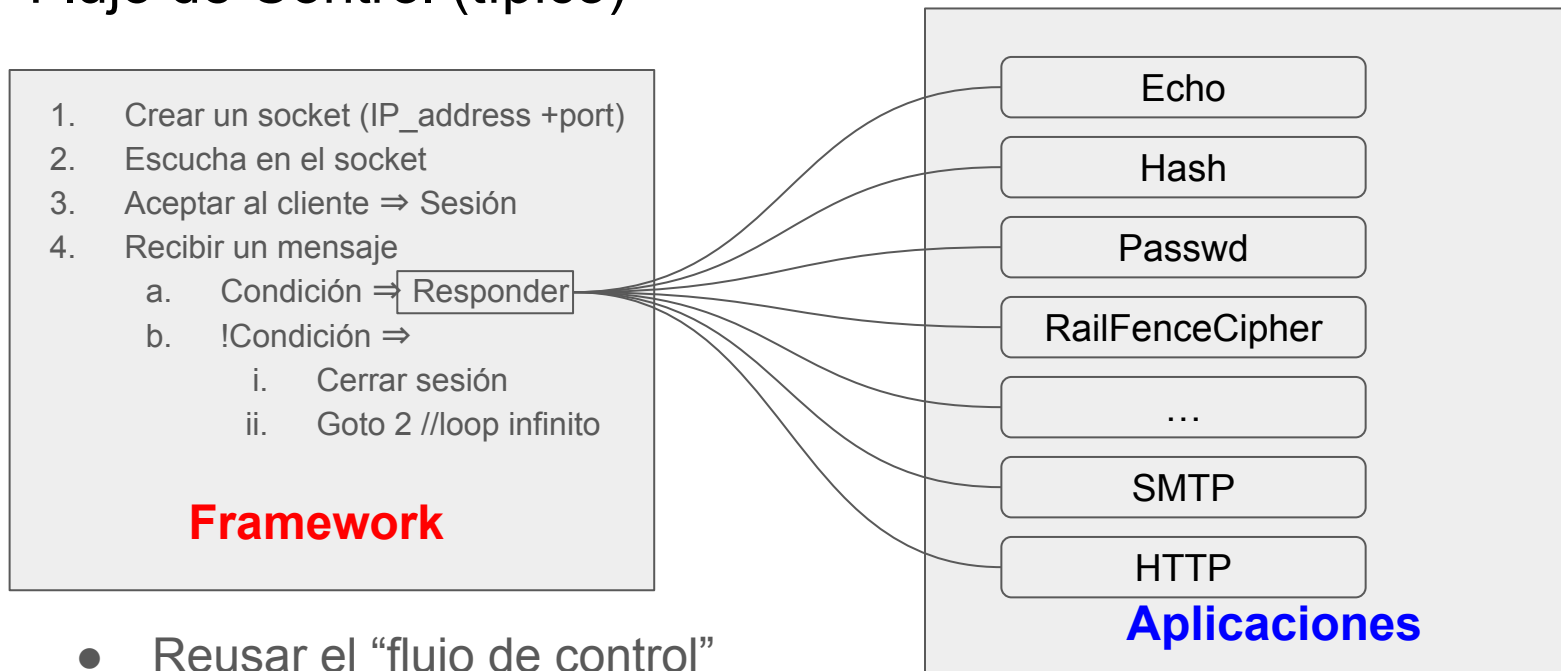
- Es un programa que implementa una de las partes de la arquitectura Cliente/Servidor
 - Un Servidor escucha en un socket TCP (IP_address+port)
 - Un Cliente establece una conexión con el servidor para enviar mensajes (plain text)
 - El Servidor responde
- Ejemplos:
 - SMTPServer: emails
 - DayTimeServer: fecha actual
 - HTTPServer: envío y recepción de documentos web
 - Backend applications
 - Servicios / Cloud microservices
 - EchoServer (helloworld de este tipo de aplicaciones):
 - Recibe un mensaje de texto
 - Responde con el mismo mensaje que el cliente envió

Flujo de Control (típico)



- Reusar el “flujo de control”
 - El flujo de control es incompleto (generalmente no compila)
 - “Responder” es donde se puede “enganchar” la funcionalidad
 - “Responder” es un hook (obligatorio)
- El flujo de control pertenece a un framework

Flujo de Control (típico)



- Reusar el “flujo de control”
 - El flujo de control es incompleto (generalmente no compila)
 - “Responder” es donde se puede “enganchar” la funcionalidad
 - “Responder” es un hook (obligatorio)
- El flujo de control pertenece a un framework


```
startLoop(args : String[])  
this.checkArguments(args);  
socket = new Socket(args[0]);  
displaySocketInformation(socket);  
while(running) {  
    handleClient(socket)  
}
```

```
handleClient(socket : Socket)  
input = socket.readStream();  
output = socket.writeStream();  
line = input.next();  
while(line != null){  
    handleMessage(line, output);  
    line = input.next();  
}
```



SimpleThreadTCPServer

```
+startLoop( :String[] ) : void  
-handleClient(:Socket) : void  
-handleMessage(String, :WriteStream) : void «abstract»  
#checkArguments(:String[]) : void  
#displaySocketInformation(:Socket) : void  
#displayAndExit(:int)
```

- handleMessage() es donde se implementa la funcionalidad
 - Antes del concepto de framework, se modificaba el código :-)
 - Con OOP + Frameworks, se pueden crear subclases
 - Cada subclase es una “instanciacion” del framework

SimpleThreadTCPServer (framework)

Flujo de Control (incompleto)

1. Crear un socket (IP_address +port)
2. Escucha en el socket
3. Aceptar al cliente ⇒ Sesión
4. Recibir un mensaje
 - a. Condición ⇒ **Responder**
 - b. !Condición ⇒ Cerrar sesión

Instanciación (Cookbook)

1. Subclasificar SimpleThreadTCPServer
 - a. Debe implementar Main(String[])
 - i. crear una instancia
 - ii. enviar método startLoop(String[])
 - b. Debe implementar handleMessage(String)
 - c. Métodos que podría implementar
 - i. acceptAndDisplaySocket(ServerSocket)
 - ii. checkArguments(String[])
 - iii. displayAndExit(int)
 - iv. displaySocketData(Socket)
 - v. displayUsage()
 - vi. displaySocketInformation()
 - vii. displayWarning()

SimpleThreadTCPServer

```
12 public final void startLoop(String[] args) {
13     checkArguments(args);
14
15     int portNumber = Integer.parseInt(args[0]);
16
17
18     try (ServerSocket serverSocket = new ServerSocket(portNumber)) {
19         displaySocketInformation(portNumber);
20         while (true) {
21             Socket clientSocket = acceptAndDisplaySocket(serverSocket);
22             handleClient(clientSocket);
23         }
24     } catch (IOException e) {
25         displayAndExit(portNumber);
26     }
27 }
```

1. Todos los servidores pueden redefinir:
 - a. checkArguments(), displaySocketInformation() y displayAndExit()

SimpleThreadTCPServer.handleClient(Socket)

```
62 private final void handleClient(Socket clientSocket) {
63
64     try (
65         PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);
66         BufferedReader in = new BufferedReader(new InputStreamReader(clientSocket.getInputStream()));) {
67         String inputLine;
68         while ((inputLine = in.readLine()) != null) {
69             System.out.println("Received message: " + inputLine + " from "
70                               + clientSocket.getInetAddress().getHostAddress() + ":" + clientSocket.getPort());
71             handleMessage(inputLine, out);
72             if (inputLine.equalsIgnoreCase("")) {
73                 break; // Client requested to close the connection
74             }
75         }
76         System.out.println("Connection closed with " + clientSocket.getInetAddress().getHostAddress() + ":"
77                           + clientSocket.getPort());
78     } catch (IOException e) {
79         System.err.println("Problem with communication with client: " + e.getMessage());
80     } finally {
81         try {
82             clientSocket.close();
83         } catch (IOException e) {
84             System.err.println("Error closing socket: " + e.getMessage());
85         }
86     }
87 }
88
89
```

1. Todos los servidores debern definir:
 - a. handleMessage(String, PrintWriter)

EchoServer.java

```
1  import java.io.PrintWriter;
2
3  public class EchoServer extends SingleThreadTCPServer {
4
5      public void handleMessage(String message, PrintWriter out) {
6          out.println(message);
7      }
8
9      Run | Debug
10     public static void main(String[] args) {
11         new EchoServer().startLoop(args);
12     }
13 }
14 }
```

1. startLoop(args) arranca el loop de control del framework
 - a. El Framework toma el control (**inversión de control**)

EchoServer.java

```
1  import java.io.PrintWriter;
2
3  public class EchoServer extends SingleThreadTCPServer {
4
5      public void handleMessage(String message, PrintWriter out) {
6          out.println(message);
7      }
8
9      Run | Debug
10     public static void main(String[] args) {
11         new EchoServer().startLoop(args);
12     }
13
14 }
```

1. startLoop(args) arranca el loop de control del framework
 - a. El Framework toma el control (**inversión de control**)
 - b. Ahora está completo porque se implementa handleMessage(...)
2. handleMessage(...) es invocado desde alguna parte del framework
 - a. Completa el loop de control
 - b. “Hollywood Principle”: no nos llames, nosotros te contactaremos
 - c. EchoServer hereda el loop de control no existe encapsulamiento
⇒ es una **Caja Blanca**
 - d. La ejecución del server depende de la correcta implementación de handleMessage() porque es parte del loop (incompleto) de control

Framework de Caja Blanca (white box framework)

- La instanciación hereda y completa el “hilo” de control
 - El hilo de control invoca el código
- Es posible que requiera agregar métodos a clases del framework
- Demanda conocimiento del código del framework

⇒ es una Caja Blanca ¹

HotSpots vs FrozenSpot

FrozenSpot: aspecto del framework que afecta a todas las instancias y que no se puede modificar (marca indeleble)

Ej: SingleThreadTCPServer cierra la conexión con el cliente si:

1. El cliente cierra la conexión
2. El cliente envía un mensaje vacío ¹

—Todas las aplicaciones de este framework tienen esta “marca”/arquitectura

```
72         if (inputLine.equalsIgnoreCase("")) {  
73             break; // Client requested to close the connection  
74         }
```

Ejercicio/Evaluación: como podría parametrizar la “condición de cierre de sesión” ?

(1) SingleThreadTCPServer.java: 72-74

HotSpots vs FrozenSpot

HotSpot: estructura en el código que permite modificar el comportamiento del framework, para instanciar o para extender.

Ej: SingleThreadTCPServer podría¹ usar una jerarquía de “políticas” para determinar cuando una sesión debe cerrarse.

- Toda instanciación deberían elegir la política de cierre de session
- Nuevas políticas de cierre de sesión se pueden agregar para “extender” el framework



(1) SingleThreadTCPServer.java: no tiene implementado esta modificación en la versión publicada.

Resumiendo SimpleThreadTCPServer

- El método `startLoop()`
 - Definición única de como ejecutan todas las instancias
 - Invoca al método `handleClient()` que invoca `handleMessage()`
 - De manera indirecta implementa “inversión de control” (es el hilo de control)
- `HandleMessage()`
 - Es un “gancho” para todas las instancias
 - Todas las instancias deben implementar ese método
- Existen otros métodos que pueden ser reimplementados por las instancias de los que existen implementaciones por defecto.
 - `acceptAndDisplaySocket(ServerSocket)`
 - `checkArguments(String[])`
 - `displayAndExit(int)`
 - `displaySocketData(Socket)`
 - `displayUsage()`
 - `displaySocketInformation()`
 - `displayWarning()`

Resumen de Frameworks

- Proveen una solución reusable para una familia de aplicaciones
- Las clases en el framework se relacionan (herencia, conocimiento, envío de mensajes) de manera que resuelven la mayor parte del problema en cuestión
- El código del framework controla/usa al código de la instanciación
- $\text{Instanciacion}(\text{Framework}) = \text{Aplicación/Solución}$
- $\text{Extension}(\text{Framework}) = \text{Framework}'$
- Tipos de Frameworks
 - Aplicación: desktop, webapps, tcpservers :-)
 - Manejo Datos: ORDB, pipelines, NRDB
 - Sistemas Distribuidos: mensajes, eventos, rpc
 - Testing: unit, web pages
- `oo2.next()`
 - Blackbox frameworks: instanciación por composición
 - Frameworks & Design Patterns



